

Servidor MAGINOT - Anexo II

Guía para acceder, trabajar y ejecutar análisis en el servidor

Dr. Cristian Cervantes

2026-04-26

Table of contents

1	MAGINOT — Manual de usuario	6
1.1	¿Qué es MAGINOT?	6
1.2	Objetivo del servidor	6
1.3	¿A quién está dirigido?	6
1.4	Alcances del manual	6
1.5	Reglas rápidas (uso responsable)	7
1.6	Buenas prácticas recomendadas	7
1.7	Ruta rápida (para usuarios con prisa)	7
2	Conexión: VPN, SSH y transferencia de archivos	8
2.1	Objetivo	8
2.2	Lo que necesitas (antes de empezar)	8
2.3	1) VPN — WireGuard (obligatorio)	9
2.3.1	Windows	9
2.3.2	macOS	9
2.3.3	Linux	9
2.4	2) Verificar que la VPN está funcionando (recomendado)	9
2.5	3) Acceso por SSH	10
2.5.1	Opción A — Terminal (OpenSSH) recomendado	10
2.6	4) Contraseña y primer cambio	10
2.6.1	Reglas de contraseña (obligatorias)	11
2.7	5) Transferencia de archivos con <code>scp</code>	11
2.7.1	Sintaxis básica (Linux y macOS)	11
2.7.2	Windows (PowerShell / Windows Terminal)	12
2.7.3	Errores comunes	12
2.7.4	Buenas prácticas	12
2.8	6) Clientes gráficos (recomendado): Termius	13
2.8.1	¿Por qué Termius?	13
3	Uso básico de Linux	14
3.1	Objetivo	14
3.2	2) Comandos esenciales para moverte	14
3.2.1	¿Dónde estoy?	14
3.2.2	Listar archivos	14
3.2.3	Cambiar de carpeta	14

3.2.4	Crear carpetas	15
3.2.5	Copiar, mover y borrar (con cuidado)	15
3.3	3) Revisar almacenamiento (disco) y tamaño de datos	15
3.3.1	Espacio libre por partición	15
3.3.2	Tamaño de una carpeta (resumen)	15
3.3.3	Encontrar lo que más pesa (rápido)	15
3.4	4) Revisar recursos disponibles (CPU y RAM)	16
3.4.1	CPU (modelo, núcleos, hilos)	16
3.4.2	RAM (memoria)	16
3.4.3	Ver procesos en tiempo real	16
3.5	5) Monitoreo básico: ver tus procesos, PID y cómo detenerlos	16
3.5.1	Ver procesos de tu usuario	16
3.5.2	Buscar un proceso por nombre	17
3.5.3	Ver el PID de un proceso específico	17
3.5.4	Detener un proceso (solo tus procesos)	17
3.5.5	Detener varios procesos por nombre (con cuidado)	17
3.6	6) Sesiones persistentes (recomendado): tmux	18
3.7	7) Permisos y propiedad (solo lo necesario)	18
3.7.1	Ver permisos	18
3.7.2	Cambiar permisos (ejemplos típicos)	18
3.8	8) Edición rápida de archivos: nano (y opcional vim)	19
3.9	9) Compresión y empaquetado	19
3.9.1	Crear un .tar.gz (carpeta completa)	19
3.9.2	Extraer un .tar.gz	19
3.9.3	Comprimir un archivo con gzip	19
3.9.4	Compresión paralela (si está instalado): pigz	20
4	Software en MAGINOT	21
4.1	Objetivo	21
4.2	Si necesitas un programa nuevo (importante)	21
4.2.1	¿Cómo solicitar software al administrador?	21
4.3	Módulos (Lmod): qué son y cómo se usan	22
4.3.1	Ver el catálogo de módulos disponibles	22
4.3.2	Cargar un módulo (micromamba)	23
4.3.3	Cargar un módulo (apptainer)	24
4.3.4	Ver módulos cargados	25
4.3.5	Descargar un módulo	25
4.3.6	Limpiar el entorno (recomendado si cambias de proyecto)	25
4.3.7	Consejos para evitar conflictos	25
4.4	Micromamba (ambientes administrados)	26
4.5	Apptainer (contenedores)	27

5	Servicios: RStudio Server	28
5.1	RStudio Server	28
5.2	1) Requisitos	28
5.3	2) Cómo conectarte	28
5.4	3) Recomendaciones de uso	29
5.5	4) Problemas comunes	29
5.5.1	No carga la página	29
5.5.2	“Connection refused” o “No se puede establecer conexión”	29
5.5.3	Login incorrecto	29
6	Uso básico de Slurm	30
6.1	¿Qué es Slurm?	30
6.2	Conceptos básicos	30
6.2.1	Job	30
6.2.2	Cola	30
6.2.3	Partición	31
6.2.4	Recursos	31
6.3	Comandos básicos	31
6.3.1	Ver el estado del servidor	31
6.3.2	Ver trabajos en cola	32
6.3.3	Ver solo tus trabajos	32
6.3.4	Cancelar un trabajo	32
6.3.5	Cancelar todos tus trabajos	33
6.4	Primer ejemplo: ejecutar un comando simple	33
6.4.1	Enviar trabajos con sbatch	33
6.5	Script mínimo de ejemplo	33
6.5.1	Revisar la salida del trabajo	34
6.5.2	Personalizar el nombre de salida	34
6.6	Parámetros principales de Slurm	35
6.7	¿Cuántas CPUs pedir?	36
6.7.1	Programa de un solo hilo	36
6.7.2	Programa con 4 hilos	36
6.7.3	Programa con 16 hilos	36
6.7.4	Ejemplo para un programa bioinformático	37
6.7.5	Ejemplo con Apptainer	37
6.8	Variables útiles de Slurm	38
6.9	Buenas prácticas	39
6.9.1	1. No correr análisis pesados directamente en la terminal	39
6.9.2	2. No pedir más CPUs de las necesarias	39
6.9.3	3. Siempre usar carpetas organizadas	39
6.9.4	4. Guardar salidas y errores	40
6.9.5	5. Probar primero con datos pequeños	40
6.9.6	6. Revisar los logs	40

6.10	Revisar trabajos completados	40
6.11	Plantilla recomendada para usuarios	40
6.11.1	Resumen de comandos esenciales	41
6.12	Recomendación final	42
7	Referencias	43
7.1	Documentación y recursos oficiales	43
7.2	Tutoriales recomendados	43
7.3	Créditos	43

1 MAGINOT — Manual de usuario

Guía para acceder, trabajar y ejecutar análisis en el servidor

1.1 ¿Qué es MAGINOT?

MAGINOT es un servidor GNU/Linux multiusuario orientado a la ejecución rápida y reproducible de pipelines bioinformáticos, con acceso seguro, almacenamiento organizado y control responsable de recursos.

1.2 Objetivo del servidor

Diseñar e implementar un servidor GNU/Linux multiusuario, seguro y observable que permita la ejecución rápida y reproducible de pipelines bioinformáticos mediante contenedores y control de recursos, con almacenamiento confiable y cuotas, para dar un servicio estable.

1.3 ¿A quién está dirigido?

A estudiantes, técnicos y personal académico que requieran ejecutar análisis bioinformáticos en un entorno compartido, siguiendo buenas prácticas de seguridad, organización y uso responsable de recursos.

1.4 Alcances del manual

Incluye - Acceso (VPN/SSH) - Navegación y organización de proyectos - Transferencia de archivos (SFTP/SCP/rsync) - Uso de software (módulos/ambientes/contenedores) - Ejecución y monitoreo básico de análisis

No incluye - Cursos completos de Linux o bioinformática - Desarrollo de pipelines desde cero - Instalación libre de software sin solicitud previa al administrador

1.5 Reglas rápidas (uso responsable)

- La cuenta es personal e intransferible.
- No compartas contraseñas.
- Mantén tus proyectos ordenados y limpia datos temporales.
- Reporta incidentes o problemas al administrador.

1.6 Buenas prácticas recomendadas

- **Organiza tus proyectos:** usa una estructura consistente (por ejemplo `data/`, `scripts/`, `results/`, `logs/`, `metadata/`).
- **Reproducibilidad:** registra versiones, parámetros y comandos (idealmente en scripts).
- **Logs:** guarda salida estándar y errores; te ahorran tiempo para depurar.
- **Almacenamiento responsable:** comprime resultados grandes y elimina temporales cuando ya no se requieran.
- **Seguridad:** usa contraseñas fuertes.

1.7 Ruta rápida (para usuarios con prisa)

1. Conéctate a la VPN (WireGuard).
2. Entra por SSH al servidor.
3. Ubica o crea tu carpeta de proyecto.
4. Sube tus datos (SFTP/rsync/scp).
5. Carga software (modules/conda) o usa contenedores (Apptainer).
6. Ejecuta tu análisis siguiendo las reglas de recursos.
7. Monitorea y guarda logs/resultados.

2 Conexión: VPN, SSH y transferencia de archivos

2.1 Objetivo

Esta sección describe lo necesario para conectarte a MAGINOT desde tu sistema operativo: **VPN (WireGuard)** + **SSH**, y (opcional) **SFTP** para transferir archivos.

2.2 Lo que necesitas (antes de empezar)

Debes contar con:

- **Usuario** en MAGINOT
- **Acceso VPN**: archivo **WireGuard .conf**
- **Datos de conexión SSH**
 - **Host** (IP)
 - **Puerto**
 - **Usuario**
- Conexión a internet

Importante: normalmente MAGINOT solo es accesible después de conectarte a la VPN.

2.3 1) VPN — WireGuard (obligatorio)

2.3.1 Windows

1. Instala **WireGuard** (Windows).
2. Abre WireGuard → **Add Tunnel** → **Import tunnel(s) from file**.
3. Selecciona tu archivo `usuario.conf`.
4. Activa el túnel (**Activate**).

2.3.2 macOS

1. Instala **WireGuard** desde la App Store.
2. Abre WireGuard → **Import tunnel(s) from file** (o “Add Tunnel”).
3. Selecciona tu `usuario.conf`.
4. Activa el túnel.

2.3.3 Linux

Dependiendo de la distribución:

- **Cliente CLI:** `wireguard-tools`
- **Cliente GUI:** WireGuard app / NetworkManager plugin (si aplica)

Flujo general: 1. Importa tu configuración `.conf` en la herramienta que uses (GUI o CLI). 2. Levanta el túnel.

2.4 2) Verificar que la VPN está funcionando (recomendado)

Con el túnel activo, valida conectividad con la IP que te indique el administrador.

Ejemplo:

IP: `192.168.50.211`

```
ping -c 3 192.168.50.211
```

Si el ping falla:

- revisa que el túnel esté activo

- revisa que importaste el `.conf` correcto
 - contacta al administrador si persiste
-

2.5 3) Acceso por SSH

Nota: La IP y el puerto es la que te indique el administrador.

2.5.1 Opción A — Terminal (OpenSSH) recomendado

macOS / Linux

Ejemplo:

IP: 192.168.50.25

Puerto: 14232

```
ssh -p 14232 usuario@192.168.50.25
```

Windows 10/11 (PowerShell o Windows Terminal)

```
ssh -p 14232 usuario@192.168.50.25
```

La primera vez, te pedirá confirmar la llave del servidor:

- escribe `yes` y Enter
-

2.6 4) Contraseña y primer cambio

Tu cuenta se crea con una **contraseña temporal**.

En tu primer inicio de sesión, el sistema te pedirá cambiarla automáticamente (no podrás continuar sin hacerlo).

Si por alguna razón no te aparece el cambio automático, puedes forzarlo manualmente con:

```
passwd usuario
```

2.6.1 Reglas de contraseña (obligatorias)

- Mínimo 10 caracteres
 - Debe incluir al menos:
 - 1 letra minúscula
 - 1 letra mayúscula
 - 1 número
 - 1 símbolo
 - No compartir credenciales (la cuenta es personal e intransferible)
-

2.7 5) Transferencia de archivos con scp

scp (Secure Copy) permite copiar archivos y carpetas **de forma segura** usando SSH. Funciona en **macOS**, **Linux** y también en **Windows (PowerShell/Windows Terminal)**.

Requisito: debes estar conectado a la **VPN (WireGuard)** y tener acceso por **SSH**.

2.7.1 Sintaxis básica (Linux y macOS)

IP: 192.168.50.25

Puerto: 14232

Copiar **archivo local** → **servidor**:

```
scp -P 14232 archivo.ext USUARIO@192.168.50.25:/ruta/destino/
```

Copiar **archivo servidor** → **local**:

```
scp -P 14232 USUARIO@192.168.50.25:/ruta/archivo.ext /ruta/local/
```

Copiar carpetas (recursivo)

Para subir una carpeta completa usa **-r**:

```
scp -r -P 14232 carpeta_proyecto/ USUARIO@192.168.50.25:/ruta/destino/
```

Para bajar una carpeta del servidor:

```
scp -r -P 14232 USUARIO@192.168.50.25:/ruta/carpeta_en_servidor/ ./
```

2.7.2 Windows (PowerShell / Windows Terminal)

En Windows 10/11, scp suele venir incluido. Los comandos son iguales:

```
scp -P 14232 archivo.ext USUARIO@192.168.50.25:~/
```

Tip de rutas en Windows: usa comillas si hay espacios:

```
scp -P 14232 "C:\Users\TuUsuario\Desktop\archivo.ext" USUARIO@192.168.50.25:~/
```

2.7.3 Errores comunes

Permission denied

- No tienes permisos de escritura en esa ruta.
- Solución: copia primero a tu HOME y luego mueve dentro del servidor (si tienes permisos), o pide acceso.

No such file or directory

- La ruta remota no existe o está mal escrita.
- Verifica con `ls` en SSH.

Se corta la transferencia

- Para archivos grandes, considera `rsync` (más robusto).

2.7.4 Buenas prácticas

- Usa rutas claras y estructura de proyecto (`data/`, `results/`, `logs/`).
- Para archivos grandes, comprime antes de transferir (`tar.gz`) o usa `rsync`.
- Evita copiar datos sensibles sin autorización.

2.8 6) Clientes gráficos (recomendado): Termius

Además de la terminal (OpenSSH) y `scp`, existen **clientes gráficos** que facilitan mucho el trabajo diario: guardan tus hosts, simplifican la autenticación, integran **SFTP** para transferencia de archivos y permiten **port forwarding** con asistentes.

Una opción que recomiendo es [Termius](#), un cliente moderno de SSH disponible para **Windows, macOS, Linux, iOS, iPadOS y Android**. Incluye funciones como “hosts”, **SFTP integrado** y **port forwarding** con un asistente paso a paso.

2.8.1 ¿Por qué Termius?

- **Multiplataforma** (desktop y móvil).
- **Organización de accesos**: guardas hosts (IP/dominio, puerto, usuario) para conectar con un clic.
- **SFTP integrado**: transferencia de archivos sin salir de la app.
- **Port forwarding** guiado: ideal para acceder a servicios como RStudio Server u otros puertos internos (cuando esté permitido).

Nota: Termius tiene planes gratuitos y de pago; algunas funciones avanzadas dependen del plan.

Buenas prácticas de seguridad (muy importante)

- Evita guardar contraseñas en equipos compartidos.

3 Uso básico de Linux

3.1 Objetivo

Esta sección te ayuda a moverte con seguridad dentro de MAGINOT: entender la estructura de carpetas, usar comandos básicos, revisar almacenamiento y recursos (CPU/RAM), y monitorear/detener procesos propios.

3.2 2) Comandos esenciales para moverte

3.2.1 ¿Dónde estoy?

```
pwd
```

3.2.2 Listar archivos

```
ls  
ls -lh  
ls -lah
```

3.2.3 Cambiar de carpeta

```
cd /ruta  
cd ~  
cd ..
```

3.2.4 Crear carpetas

```
mkdir proyecto  
mkdir -p proyecto/{data,scripts,results,logs,metadata}
```

3.2.5 Copiar, mover y borrar (con cuidado)

```
cp archivoA archivoB  
cp -r carpetaA carpetaB  
  
mv viejo.txt nuevo.txt  
mv carpeta /otra/ruta/  
  
rm archivo.txt  
rm -r carpeta/
```

rm borra (no hay papelera). Antes de borrar, confirma con ls.

3.3 3) Revisar almacenamiento (disco) y tamaño de datos

3.3.1 Espacio libre por partición

```
df -h
```

3.3.2 Tamaño de una carpeta (resumen)

```
du -sh carpeta/  
du -sh *
```

3.3.3 Encontrar lo que más pesa (rápido)

```
du -sh * | sort -h
```

Buen hábito: revisa tu uso de disco antes de lanzar análisis grandes.

3.4 4) Revisar recursos disponibles (CPU y RAM)

3.4.1 CPU (modelo, núcleos, hilos)

```
lscpu
```

3.4.2 RAM (memoria)

```
free -h
```

3.4.3 Ver procesos en tiempo real

```
top
```

O htop es más cómodo:

```
htop
```

3.5 5) Monitoreo básico: ver tus procesos, PID y cómo detenerlos

3.5.1 Ver procesos de tu usuario

```
ps -u $USER
```

Más legible:

```
ps -u $USER -o pid,ppid,cmd,%cpu,%mem,etime --sort=-%cpu | head -n 20
```

3.5.2 Buscar un proceso por nombre

```
pgrep -a nombre_programa
```

Ejemplo:

```
pgrep -a mafft
```

3.5.3 Ver el PID de un proceso específico

```
pidof nombre_programa
```

3.5.4 Detener un proceso (solo tus procesos)

Primero intenta un “apagado limpio”:

```
kill PID
```

Si no se detiene, fuerza:

```
kill -9 PID
```

Recomendación: usa `kill` normal primero. `kill -9` es el último recurso.

3.5.5 Detener varios procesos por nombre (con cuidado)

```
pkill nombre_programa
```

Verifica antes con:

```
pgrep -a nombre_programa
```

3.6 6) Sesiones persistentes (recomendado): tmux

Si tu conexión se corta, una sesión persistente evita perder tu trabajo.

Crear una sesión:

```
tmux new -s mi_sesion
```

Salir sin cerrar la sesión:

- Presiona Ctrl+b y luego d

Volver a entrar:

```
tmux attach -t mi_sesion
```

Listar sesiones:

```
tmux ls
```

3.7 7) Permisos y propiedad (solo lo necesario)

3.7.1 Ver permisos

```
ls -l
```

3.7.2 Cambiar permisos (ejemplos típicos)

Dar permiso de ejecución a un script:

```
chmod +x script.sh
```

Permisos comunes:

- 644 archivos (lectura para todos, escritura solo dueño)
- 755 ejecutables o carpetas (dueño escribe, otros leen/entran)

Ejemplos:

```
chmod 644 archivo.txt  
chmod 755 carpeta/
```

3.8 8) Edición rápida de archivos: nano (y opcional vim)

Abrir un archivo:

```
nano archivo.txt
```

Atajos útiles:

- Guardar: Ctrl+O (Enter)
 - Salir: Ctrl+X
 - Buscar: Ctrl+W
-

3.9 9) Compresión y empaquetado

3.9.1 Crear un .tar.gz (carpeta completa)

```
tar -czvf proyecto.tar.gz proyecto/
```

3.9.2 Extraer un .tar.gz

```
tar -xzf proyecto.tar.gz
```

3.9.3 Comprimir un archivo con gzip

```
gzip archivo.txt  
gunzip archivo.txt.gz
```

3.9.4 Compresión paralela (si está instalado): pigz

pigz es como gzip pero usa varios núcleos.

Comprimir:

```
pigz archivo_grande.fastq
```

Descomprimir:

```
unpigz archivo_grande.fastq.gz
```

4 Software en MAGINOT

4.1 Objetivo

En MAGINOT el software se ofrece de forma controlada para mantener un entorno ****estable, seguro y reproducible****.

Esta sección explica cómo usar el sistema de **módulos (Lmod)** y, al final, cómo se gestionan herramientas vía **Micromamba** y **Apptainer**.

4.2 Si necesitas un programa nuevo (importante)

Si el software que necesitas **no está disponible** como módulo o contenedor, **no intentes instalarlo por tu cuenta**.

En MAGINOT, la instalación de software que requiere permisos (y la creación/actualización de ambientes) se realiza **solo por el administrador**.

4.2.1 ¿Cómo solicitar software al administrador?

Incluye en tu solicitud:

- **Nombre del software**
- **Versión** (si aplica)
- **Uso** (curso/proyecto)
- **Enlace oficial** (GitHub/docs) o referencia de instalación (conda/bioconda, contenedor, etc.)
- Requerimientos especiales: **MPI**, **GPU**, bases de datos externas, etc.

Cuando el software esté disponible, el administrador te avisará y te indicará si se usa como:

- **módulo**,
- **contenedor Apptainer**,
- o dentro de un **ambiente Micromamba**.

4.3 Módulos (Lmod): qué son y cómo se usan

Los **módulos** permiten activar software instalado en el servidor sin instalar nada en tu cuenta. Al cargar un módulo, se ajustan variables de entorno (por ejemplo `PATH`) para que el comando quede disponible.

4.3.1 Ver el catálogo de módulos disponibles

```
module spider
```

```
stoich@maginot:~$ module spider

-----
The following is a list of the modules and extensions currently available:
-----
edta: edta/2.0.1

lmod: lmod
      Lmod: An Environment Module System

repeatmasker: repeatmasker/4.2.2

repeatmodeler: repeatmodeler/2.0.7

revbayes: revbayes/1.2.5-mpi

samtools: samtools/1.19

settag: settag
        The settag module provides a way to connect the loaded modules with your build system by setting environment variables.

-----

To learn more about a package execute:

    $ module spider Foo

where "Foo" is the name of a module.

To find detailed information about a particular package you
must specify the version if there is more than one version:

    $ module spider Foo/11.1

-----
```

En este momento (ejemplo real en MAGINOT) aparecen módulos como:

- edta/2.0.1

- repeatmasker/4.2.2
- repeatmodeler/2.0.7
- revbayes/1.2.5-mpi
- samtools/1.19

4.3.2 Cargar un módulo (micromamba)

Buscar información de un módulo

```
module spider samtools
```

```
stoich@maginot:~$ module spider samtools
-----
samtools: samtools/1.19
-----

This module can be loaded directly: module load samtools/1.19

Help:
  Samtools 1.19 (Micromamba env)
```

Cargar un módulo

```
module load samtools/1.19
```

Verifica:

```
which samtools
samtools --version
```

```
stoich@maginot:~$ which samtools
/opt/micromamba/envs/samtools-1.19/bin/samtools
stoich@maginot:~$ samtools --version
samtools 1.19.2
Using htlib 1.21
Copyright (C) 2024 Genome Research Ltd.
```

4.3.3 Cargar un módulo (apptainer)

Buscar información de un módulo

```
module spider revbayes
```

```
stoich@maginot:~$ module spider revbayes

-----
revbayes: revbayes/1.2.5-mpi
-----

This module can be loaded directly: module load revbayes/1.2.5-mpi

Help:
  RevBayes 1.2.5 (MPI) via Apptainer.

Este módulo expone:
- rb-mpi      : ejecuta rb-mpi (1 proceso) en el directorio actual.
- rb-mpi-np N : ejecuta rb-mpi con mpirun -np N (multinúcleo) en el directorio actual.

Ejemplo típico:
  cd /ruta/al/proyecto/Clade_001/input
  module load revbayes/1.2.5-mpi
  rb-mpi-np 8 Clade_001.rev

Binds extra (por ejemplo /srv):
  export APPTAINER_BINDPATH="/srv:/srv"
```

El módulo **revbayes/1.2.5-mpi** proporciona **RevBayes 1.2.5 con soporte MPI**, ejecutándose **dentro de un contenedor Apptainer**, pero tú lo usas como un módulo normal.

Este módulo expone dos comandos principales:

- **rb-mpi**
Ejecuta RevBayes (1 proceso) en el directorio actual.
- **rb-mpi-np N**
Ejecuta RevBayes usando `mpirun -np N` (multinúcleo) en el directorio actual.

Cargar un módulo

```
module load revbayes/1.2.5-mpi
```

Verifica:

```
which rb-mpi
rb-mpi --version
```

```
stoich@maginot:~$ which rb-mpi
rb-mpi ()
{
    aptainer exec --cleanenv --bind "$PWD":/work --pwd /work /opt/containers/revbayes/rb_v125_mpi/revbayes_v125_mpi.sif rb-mpi "$@"
}
stoich@maginot:~$ rb-mpi --version
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
1.2.5
stoich@maginot:~$
```

Ejemplo de ejecución:

```
cd /ruta/al/proyecto/Clade_001/input
module load revbayes/1.2.5-mpi
rb-mpi-np 8 Clade_001.rev
```

4.3.4 Ver módulos cargados

```
module list
```

4.3.5 Descargar un módulo

```
module unload samtools/1.19
```

4.3.6 Limpiar el entorno (recomendado si cambias de proyecto)

```
module purge
```

4.3.7 Consejos para evitar conflictos

- Inicia con `module purge` si vienes de otra sesión/proyecto.
- Si algo “no corre”, revisa primero `module list` y confirma versiones.
- Si necesitas otra versión, pide al administrador que la agregue como módulo.

4.4 Micromamba (ambientes administrados)

Micromamba se usa para mantener ambientes controlados y reproducibles, pero en MAG-INOT:

Solo el administrador puede:

- crear nuevos ambientes
- instalar/actualizar paquetes dentro de los ambientes

Los usuarios **no crean** ambientes ni instalan paquetes.

Si necesitas un programa por esta vía, **solicítalo al administrador**.

Cuando esté disponible, se te indicará cómo usarlo.

Ver ambientes disponibles

```
micromamba env list
```

Activar (cargar) un ambiente

```
micromamba activate NOMBRE_DEL_AMBIENTE
```

Verifica que el programa ya esté disponible:

```
which programa  
programa --version
```

Ver paquetes instalados en el ambiente

```
micromamba list
```

Salir del ambiente

```
micromamba deactivate
```

4.5 Apptainer (contenedores)

Apptainer es una herramienta de contenedores diseñada para entornos Linux multiusuario (común en HPC).

Permite ejecutar software empaquetado en una imagen (por ejemplo `.sif`) de forma **re-productible**, sin tener que instalar dependencias en tu cuenta, y sin requerir privilegios de administrador para correrlo.

En MAGINOT, Apptainer ya está instalado y puede usarse para correr herramientas bioinformáticas dentro de contenedores (propios o institucionales), siempre que tengas permisos de lectura sobre la imagen.

Ejemplos:

```
apptainer exec imagen.sif comando --help
```

Montar tu carpeta de trabajo dentro del contenedor:

```
apptainer exec -B "$PWD":/work --pwd /work imagen.sif comando
```

Si necesitas una guía más completa sobre **Apptainer**, consulta mi tutorial: <https://cristoichkov.github.io/biocontainers/>

5 Servicios: RStudio Server

5.1 RStudio Server

MAGINOT puede ofrecer **RStudio Server** como servicio web para trabajar en R desde el navegador.

Como el acceso es por **VPN (WireGuard)**, normalmente **no es necesario usar port forwarding**: basta con conectarte a la IP interna asignada y el puerto del servicio.

5.2 1) Requisitos

- VPN (WireGuard) **activa**
 - IP o dominio interno del servidor (proporcionado por el administrador)
 - **Puerto de RStudio Server** (proporcionado por el administrador)
 - Credenciales de tu cuenta de MAGINOT (usuario/contraseña)
-

5.3 2) Cómo conectarte

1. Conéctate a la **VPN**.
2. Abre tu navegador (Chrome/Firefox/Safari).
3. En la barra de direcciones escribe:

10.8.0.1:8787

IP: 10.8.0.1

Puerto: 8787

4. Inicia sesión con tu **usuario y contraseña** de MAGINOT.
-

5.4 3) Recomendaciones de uso

- Usa RStudio para análisis interactivos, edición de scripts y revisión de resultados.
 - Para análisis largos o pesados, sigue las reglas del servidor (por ejemplo, ejecutarlos de forma controlada y guardando logs).
 - Mantén tus proyectos organizados (por ejemplo `data/`, `scripts/`, `results/`, `logs/`).
-

5.5 4) Problemas comunes

5.5.1 No carga la página

- Verifica que la VPN esté activa.
- Confirma que estás usando la **IP correcta** y el **puerto correcto**.
- Prueba conectividad:

```
ping -c 3 10.8.0.1
```

5.5.2 “Connection refused” o “No se puede establecer conexión”

- El servicio puede estar detenido o el puerto cambió.
- Contacta al administrador.

5.5.3 Login incorrecto

- Asegúrate de usar tu usuario de MAGINOT.
- Si tu contraseña expiró, entra primero por SSH para cambiarla y vuelve a intentar.

6 Uso básico de Slurm

6.1 ¿Qué es Slurm?

Slurm es un sistema de administración de trabajos para servidores de cómputo. Su función es organizar el uso de los recursos del servidor, como CPUs, memoria y tiempo de ejecución.

En lugar de ejecutar análisis directamente en la terminal, los usuarios envían sus trabajos a una cola. Slurm decide cuándo correrlos de acuerdo con los recursos disponibles.

Esto ayuda a:

- evitar que una sola persona sature el servidor;
- ordenar los análisis de varios usuarios;
- asignar CPUs de manera controlada;
- dejar registro de los trabajos ejecutados;
- mejorar la reproducibilidad de los análisis.

6.2 Conceptos básicos

6.2.1 Job

Un *job* es un trabajo enviado a Slurm. Puede ser un script de Bash que ejecuta un análisis bioinformático, un programa de R, Python, Apptainer, BLAST, IQ-TREE, MAFFT, etc.

6.2.2 Cola

La cola es la lista de trabajos esperando o ejecutándose.

6.2.3 Partición

Una partición es una sección lógica del servidor. En MAGINOT la partición principal se llama:

```
ripley
```

significa:

“enviar mi trabajo a la cola llamada `ripley`”

6.2.4 Recursos

Cuando envías un trabajo, debes indicar cuántos recursos necesitas. Por ejemplo:

- número de CPUs;
- memoria;
- tiempo máximo;
- nombre del trabajo;
- partición donde se ejecutará.

6.3 Comandos básicos

6.3.1 Ver el estado del servidor

```
sinfo
```

Salida esperada:

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
ripley*	up	infinite	1	idle	maginot

Estados comunes:

Estado	Significado
idle	El nodo está libre
alloc	El nodo está ocupado
mix	El nodo tiene algunos recursos ocupados y otros libres
down	El nodo no está disponible

Estado	Significado
drain	El nodo no acepta nuevos trabajos

6.3.2 Ver trabajos en cola

```
squeue
```

Ejemplo:

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
1234	ripley	analisis	usuario	R	1:23	1	maginot

Estados comunes de trabajos:

Estado	Significado
R	Running, el trabajo está corriendo
PD	Pending, el trabajo está esperando
CG	Completing, el trabajo está terminando
CD	Completed, el trabajo terminó correctamente
F	Failed, el trabajo falló
CA	Cancelled, el trabajo fue cancelado

6.3.3 Ver solo tus trabajos

```
squeue -u $USER
```

6.3.4 Cancelar un trabajo

```
scancel JOBID
```

Ejemplo:

```
scancel 1234
```

6.3.5 Cancelar todos tus trabajos

Usar con cuidado:

```
scancel -u $USER
```

6.4 Primer ejemplo: ejecutar un comando simple

Puedes probar Slurm con:

```
srun -p riple -N1 -n1 hostname
```

La salida esperada es:

```
maginot
```

Esto significa que Slurm ejecutó el comando en el servidor.

6.4.1 Enviar trabajos con sbatch

La forma recomendada de usar Slurm es crear un script y enviarlo con:

```
sbatch script.sh
```

Un script de Slurm es un archivo de Bash con instrucciones especiales que empiezan con:

```
#SBATCH
```

Estas líneas le dicen a Slurm qué recursos necesitas.

6.5 Script mínimo de ejemplo

Crear un archivo llamado `prueba_slurm.sh`:

```
nano prueba_slurm.sh
```

Contenido:

```
#!/bin/bash
#SBATCH -J prueba
#SBATCH -p riple
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 2
#SBATCH --time=00:05:00

echo "Inicio: $(date)"
echo "Host: $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "CPUs asignadas: $SLURM_CPUS_PER_TASK"

sleep 10

echo "Fin: $(date)"
```

Enviar:

```
sbatch prueba_slurm.sh
```

Slurm responderá algo como:

```
Submitted batch job 1234
```

6.5.1 Revisar la salida del trabajo

Por defecto, Slurm crea un archivo:

```
slurm-JOBID.out
```

Por ejemplo:

```
cat slurm-1234.out
```

6.5.2 Personalizar el nombre de salida

Es recomendable guardar las salidas en una carpeta.

Crear carpeta:

```
mkdir -p logs
```

Ejemplo de script:

```
#!/bin/bash
#SBATCH -J prueba
#SBATCH -p riple
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 2
#SBATCH --time=00:05:00
#SBATCH -o logs/%x_%j.out
#SBATCH -e logs/%x_%j.err

echo "Inicio: $(date)"
echo "Host: $(hostname)"
echo "Job ID: $SLURM_JOB_ID"
echo "Fin: $(date)"
```

Donde:

Código	Significado
%x	Nombre del trabajo
%j	ID del trabajo

Así se generan archivos como:

```
logs/prueba_1234.out logs/prueba_1234.err
```

6.6 Parámetros principales de Slurm

Opción	Significado
#SBATCH -J nombre	Nombre del trabajo
#SBATCH -p riple	Partición donde se ejecuta
#SBATCH -N 1	Número de nodos
#SBATCH -n 1	Número de tareas
#SBATCH -c 4	CPUs por tarea
#SBATCH --mem=16G	Memoria solicitada
#SBATCH --time=01:00:00	Tiempo máximo
#SBATCH -o archivo.out	Archivo de salida estándar
#SBATCH -e archivo.err	Archivo de error

6.7 ¿Cuántas CPUs pedir?

No siempre conviene pedir muchas CPUs. Solo deben solicitarse si el programa realmente puede usarlas.

Ejemplos:

6.7.1 Programa de un solo hilo

```
#SBATCH -c 1
```

6.7.2 Programa con 4 hilos

```
#SBATCH -c 4
```

6.7.3 Programa con 16 hilos

```
#SBATCH -c 16
```

Además, dentro del comando del programa hay que indicar el mismo número de hilos.

Ejemplo con fastp:

```
fastp \  
-i muestra_R1.fastq.gz \  
-I muestra_R2.fastq.gz \  
-o limpia_R1.fastq.gz \  
-O limpia_R2.fastq.gz \  
-w $SLURM_CPUS_PER_TASK
```

Ejemplo con iqtree2:

```
iqtree2 -s alineamiento.phy -T $SLURM_CPUS_PER_TASK
```

Ejemplo con mafft:

```
mafft --thread $SLURM_CPUS_PER_TASK entrada.fasta > alineado.fasta
```

6.7.4 Ejemplo para un programa bioinformático

```
#!/bin/bash
#SBATCH -J mafft_test
#SBATCH -p riple
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 8
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH -o logs/%x_%j.out
#SBATCH -e logs/%x_%j.err

mkdir -p logs resultados

echo "Inicio: $(date)"
echo "Host: $(hostname)"
echo "CPUs: $SLURM_CPUS_PER_TASK"

mafft --thread $SLURM_CPUS_PER_TASK datos/secuencias.fasta >
    resultados/secuencias_alineadas.fasta

echo "Fin: $(date)"
```

Enviar:

```
sbatch mafft_test.sh
```

6.7.5 Ejemplo con Apptainer

Si el análisis usa un contenedor de Apptainer:

```
#!/bin/bash
#SBATCH -J apptainer_test
#SBATCH -p riple
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 4
#SBATCH --mem=8G
#SBATCH --time=01:00:00
#SBATCH -o logs/%x_%j.out
#SBATCH -e logs/%x_%j.err

mkdir -p logs resultados
```

```

echo "Inicio: $(date)"
echo "Host: $(hostname)"

apptainer exec programa.sif programa --help

echo "Fin: $(date)"

```

Ejemplo con carpetas enlazadas:

```

#!/bin/bash
#SBATCH -J apptainer_test
#SBATCH -p rpley
#SBATCH -N 1
#SBATCH -n 1
#SBATCH -c 4
#SBATCH --mem=8G
#SBATCH --time=01:00:00
#SBATCH -o logs/%x_%j.out
#SBATCH -e logs/%x_%j.err

mkdir -p logs resultados

echo "Inicio: $(date)"
echo "Host: $(hostname)"

apptainer exec \
  --bind $PWD:/work \
  --pwd /work \
  programa.sif \
  programa input.fasta output.txt

echo "Fin: $(date)"

```

6.8 Variables útiles de Slurm

Dentro de un script, Slurm crea variables automáticamente.

Variable	Significado
<code>\$SLURM_JOB_ID</code>	ID del trabajo
<code>\$SLURM_JOB_NAME</code>	Nombre del trabajo
<code>\$SLURM_CPUS_PER_TASK</code>	CPUs asignadas
<code>\$SLURM_SUBMIT_DIR</code>	Carpeta desde donde se envió el job
<code>\$SLURM_JOB_NODELIST</code>	Nodo asignado

Ejemplo:

```
echo "Job ID: $SLURM_JOB_ID"  
echo "Nombre: $SLURM_JOB_NAME"  
echo "CPUs: $SLURM_CPUS_PER_TASK"  
echo "Directorio de envío: $SLURM_SUBMIT_DIR"
```

6.9 Buenas prácticas

6.9.1 1. No correr análisis pesados directamente en la terminal

Evita ejecutar programas pesados así:

```
iqtree2 -s archivo.phy -T 72
```

En su lugar, crea un script y envíalo con:

```
sbatch script.sh
```

6.9.2 2. No pedir más CPUs de las necesarias

Si un programa solo usa 4 hilos, no pidas 32 CPUs.

Mal:

```
#SBATCH -c 32
```

Bien:

```
#SBATCH -c 4
```

6.9.3 3. Siempre usar carpetas organizadas

Ejemplo:

```
proyecto/  
  datos/  
  scripts/  
  resultados/  
  logs/
```

6.9.4 4. Guardar salidas y errores

Usa:

```
#SBATCH -o logs/%x_%j.out  
#SBATCH -e logs/%x_%j.err
```

6.9.5 5. Probar primero con datos pequeños

Antes de enviar un análisis grande, prueba con un subconjunto.

6.9.6 6. Revisar los logs

Si un trabajo falla, revisa primero:

```
cat logs/nombre_JOBID.err  
cat logs/nombre_JOBID.out
```

6.10 Revisar trabajos completados

El servidor guarda un registro básico de trabajos completados en:

```
/srv/bishop/slurm/log/job_completions.log
```

Puedes revisar los últimos trabajos con:

```
tail -n 20 /srv/bishop/slurm/log/job_completions.log
```

6.11 Plantilla recomendada para usuarios

```
#!/bin/bash  
#SBATCH -J nombre_del_analisis  
#SBATCH -p ripleys  
#SBATCH -N 1  
#SBATCH -n 1  
#SBATCH -c 4  
#SBATCH --mem=16G  
#SBATCH --time=02:00:00  
#SBATCH -o logs/%x_%j.out
```

```

#SBATCH -e logs/%x_%j.err

set -euo pipefail

mkdir -p logs resultados

echo "====="
echo "Trabajo: $SLURM_JOB_NAME"
echo "Job ID: $SLURM_JOB_ID"
echo "Usuario: $USER"
echo "Host: $(hostname)"
echo "Inicio: $(date)"
echo "CPUs asignadas: $SLURM_CPUS_PER_TASK"
echo "Directorio: $(pwd)"
echo "====="

# Cargar ambiente, módulo o contenedor aquí
# module load programa
# micromamba activate ambiente
# apptainer exec programa.sif programa

# Comando principal
echo "Aquí va el análisis"

echo "====="
echo "Fin: $(date)"
echo "====="

```

6.11.1 Resumen de comandos esenciales

Acción	Comando
Ver estado del servidor	sinfo
Ver cola	squeue
Ver tus trabajos	squeue -u \$USER
Enviar trabajo	sbatch script.sh
Cancelar trabajo	scancel JOBID
Ejecutar comando simple	srun -p ripley -N1 -n1 hostname
Ver últimos trabajos registrados	tail -n 20 /srv/bishop/slurm/log/job_completions.log

6.12 Recomendación final

Antes de enviar un análisis grande:

1. Revisa que tu script funcione con pocos datos.
2. Pide solo los recursos necesarios.
3. Guarda salidas y errores en `logs/`.
4. Verifica la cola con `squeue`.
5. Revisa los archivos `.out` y `.err`.

El uso correcto de Slurm permite que MAGINOT sea utilizado por varias personas de forma ordenada, evitando saturar el servidor y facilitando el seguimiento de los análisis.

7 Referencias

Esta página reúne enlaces y recursos citados o recomendados a lo largo del manual.

7.1 Documentación y recursos oficiales

- **WireGuard** — **Sitio oficial:** <https://www.wireguard.com/>
- **OpenSSH (cliente SSH):** <https://www.openssh.com/>
- **RStudio Server (Posit):** <https://posit.co/products/open-source/rstudio-server/>
- **Lmod (Environment Modules):** <https://lmod.readthedocs.io/>
- **Apptainer (documentación):** <https://apptainer.org/docs/>
- **Micromamba:** https://mamba.readthedocs.io/en/latest/user_guide/micromamba.html

7.2 Tutoriales recomendados

- **Biocontainers / Apptainer (tutorial):** <https://cristoichkov.github.io/biocontainers/>

7.3 Créditos

Este manual fue elaborado para apoyar el uso responsable del servidor MAGINOT.